



# Agri-sense

**Microdosing AI brief  
project**

**By: Omar Yussuf**



# Introduction

AgriSense is my response to the TeleAgriCulture live brief. It's an interpretable AI pipeline that transforms raw environmental sensor readings from the teleagriculture boards into actionable, distilled climate insights merged with OpenWeather for community growers and collaborators.



Minodu





# My Framing of the Problem

## The Gap

- TeleAgriCulture collects valuable sensor data
- Raw data is difficult for non-technical users to interpret
- No forecasting or contextualised decision support

## Opportunity

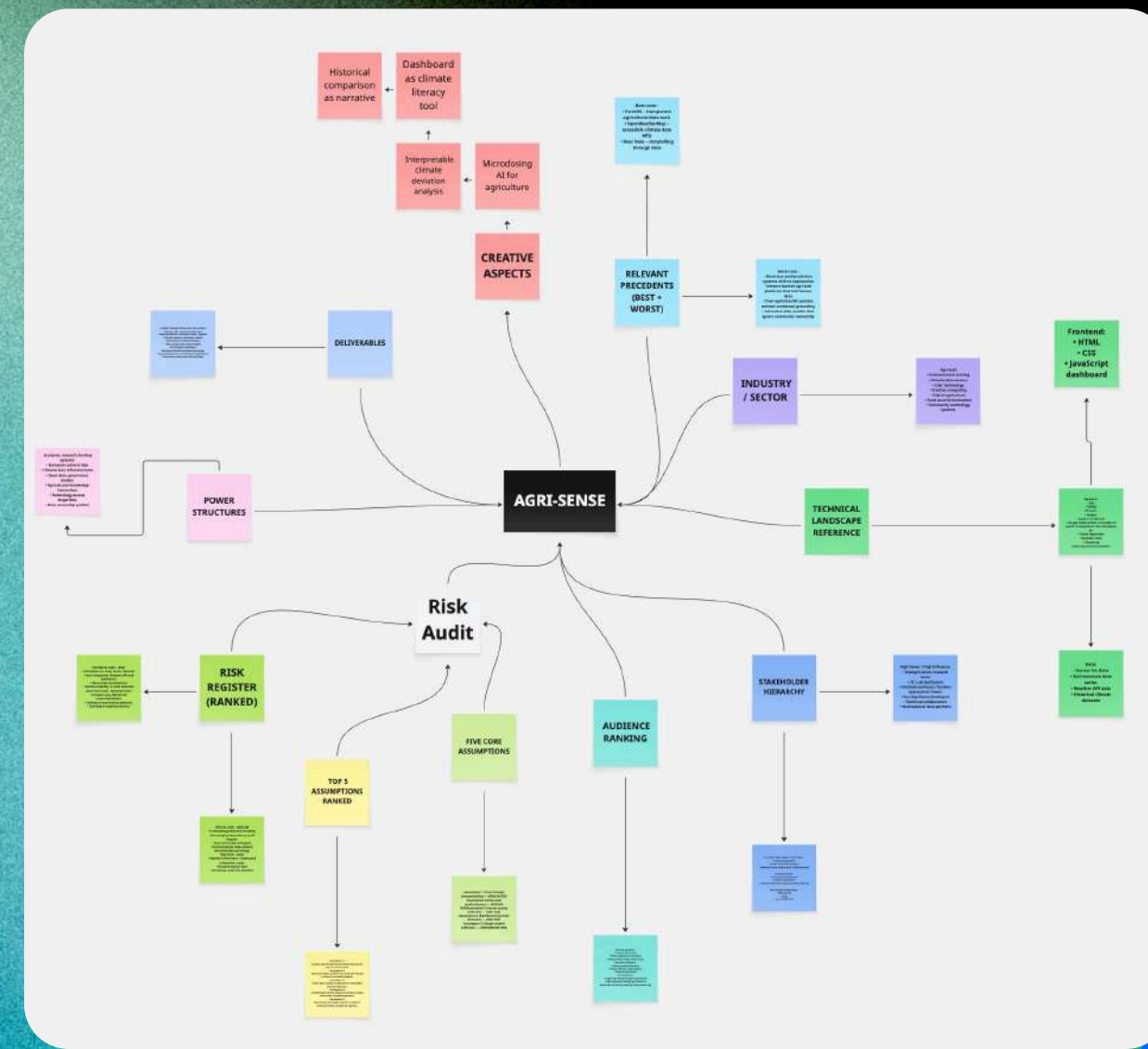
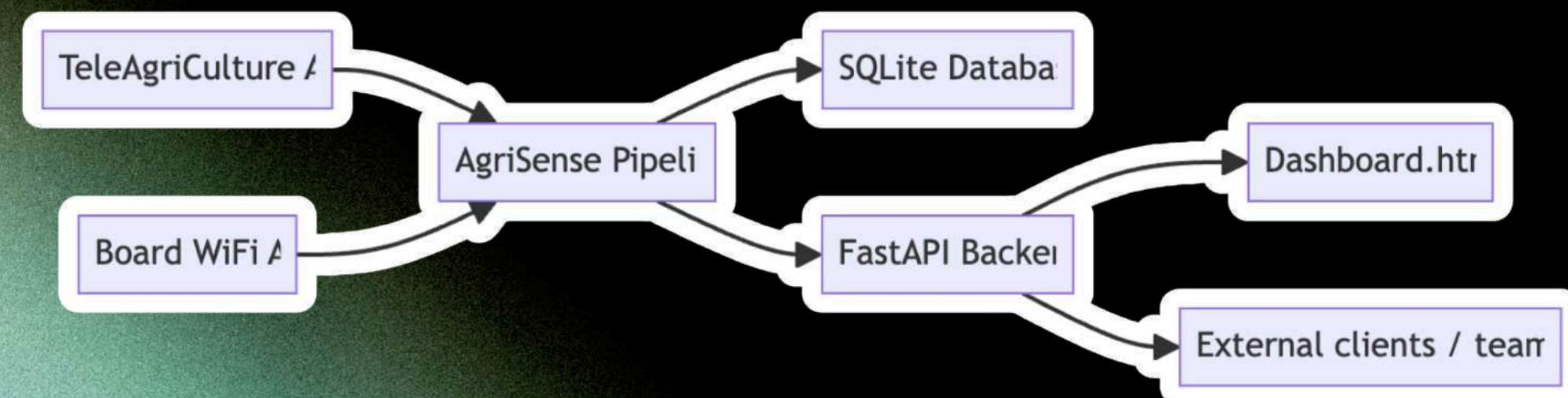
- Build a transparent insight system that:
- classifies agronomic states
- detects anomalies
- forecasts risk
- produces plain-language summaries



# Concept: What AgriSense Is

AgriSense = Insight Pipeline + API + Dashboard:

- Pulls readings from any TeleAgriCulture Kit ID
- Stores them locally in SQLite
- Runs interpretable ML
- Generates insights + alerts + forecasts
- Surfaces everything via REST API + HTML dashboard





# Interpretable AI (Microdosing in Practice)

## Agronomic Classification

### - Shallow Decision Tree (max depth 4)

- States:
  - optimal
  - heat stress
  - cold stress
  - drought risk
  - waterlogged

```
env.example  pipeline.py M X  dashboard.html  upload.py  .env
pipeline > pipeline.py > label_value
447 def build_classifier(
450     """
451     Train a shallow (max_depth=4) interpretable Decision Tree classifier.
452
453     Process:
454     1. Generate rule-based labels using THRESHOLDS (label_value)
455     2. Encode labels with LabelEncoder
456     3. Train DecisionTreeClassifier on featurised data
457
458     max_depth=4 means the tree has at most 15 decision nodes, making
459     every classification rule human-readable via export_text().
460
461     Args:
462     | df:         cleaned DataFrame
463     | sensor_name: used to select correct thresholds
464
465     Returns:
466     | clf:         trained DecisionTreeClassifier
467     | le:         fitted LabelEncoder (needed to decode predictions)
468     | feature_names: list of feature column names (for explainability)
469     """
470     X, _ = featurise(df)
471     labels = (
472         df["value"]
473         .iloc[len(df) - len(X):]
474         .apply(lambda v: label_value(v, sensor_name))
475         .values
476     )
477     le = LabelEncoder().fit(labels)
478     clf = DecisionTreeClassifier(
479         max_depth=4,
480         min_samples_leaf=5,
481         random_state=42,
482     ).fit(X, le.transform(labels))
483
484     return clf, le, list(X.columns)
485
486
```



# Anomaly Detection + Peer Benchmarking

## Real-Time Insight

- Rolling z-score anomaly detection
- Board baseline learning
- Cross-board percentile benchmarking
- Deviation from OpenWeatherMap outdoor baseline



Z-Scores

$$z = \frac{x - m}{s}$$

$x \rightarrow z$

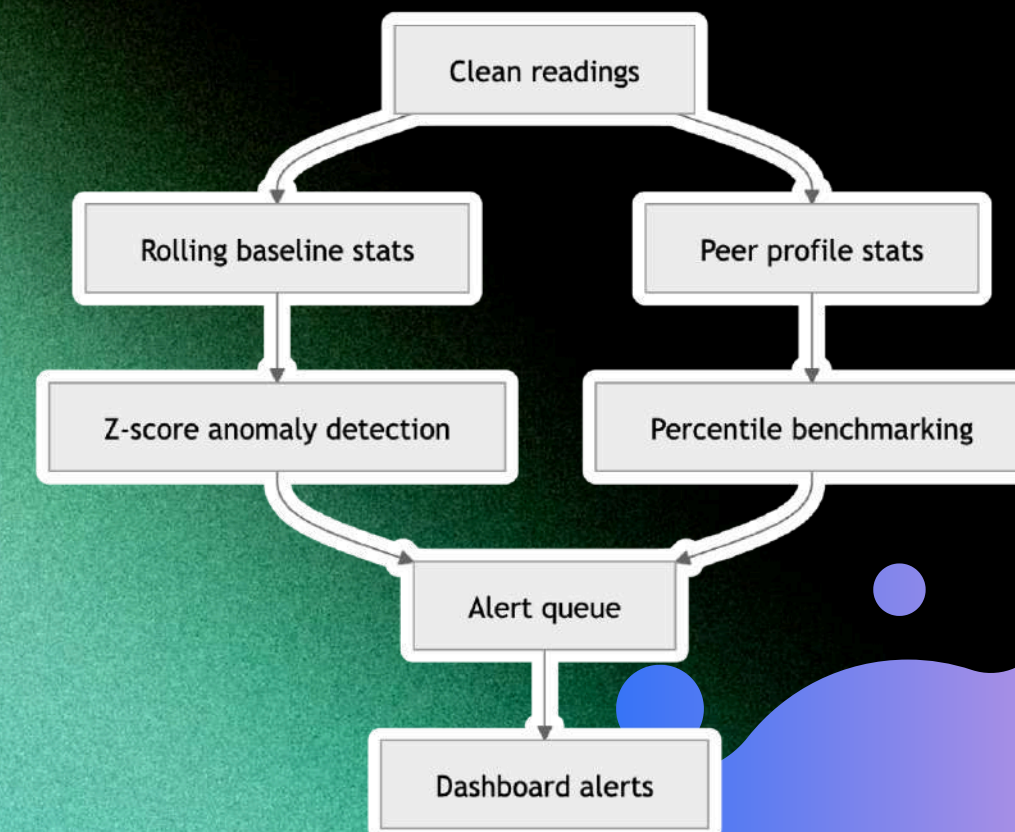
$$x = zs + m$$

$$z = \frac{2 - 4.6}{1.51} = \frac{-2.6}{1.51} = -1.73$$

$$z = 2$$
$$x = 2 \cdot 1.51 + 4.6$$

```
env.example  pipeline.py M x agrisense.db dashboard.html .env
pipeline > pipeline.py > ...

532
533 #
534 # 5. ANOMALY DETECTION
535 #
536
537 def detect_anomalies(
538     df: pd.DataFrame,
539     z_threshold: float = ANOMALY_Z_THRESHOLD,
540     window: int = ANOMALY_WINDOW,
541 ) -> pd.DataFrame:
542     """
543     Flag statistically anomalous readings using a rolling z-score.
544
545     A reading is flagged when:
546     |value - rolling_mean| / rolling_std > z_threshold
547
548     z_threshold=2.5 is tuned to avoid false positives from normal
549     diurnal cycles (day/night temperature swings etc.).
550
551     Adds columns: rolling_mean, rolling_std, z_score, is_anomaly
552     """
553     df = df.copy()
554     df["rolling_mean"] = df["value"].rolling(window=window, min_periods=3).mean()
555     df["rolling_std"] = df["value"].rolling(window=window, min_periods=3).std()
556     df["z_score"] = (
557         (df["value"] - df["rolling_mean"]) / df["rolling_std"]
558     ).abs()
559     df["is_anomaly"] = df["z_score"] > z_threshold
560     return df
561
562
563 def build_alert(
564     reading: dict,
565     sensor_name: str,
566     z_score: float,
567     board_mean: float,
568     kit_id: str,
569 ) -> dict:
```

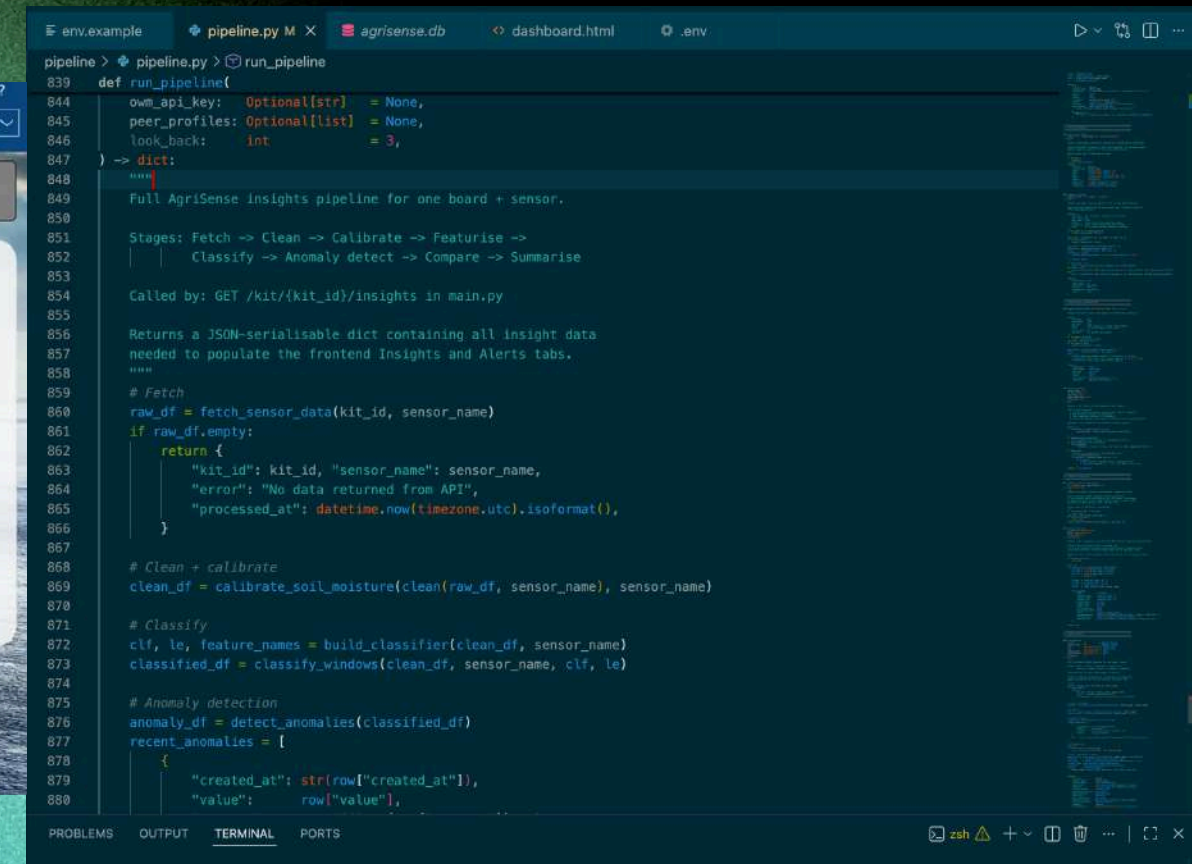
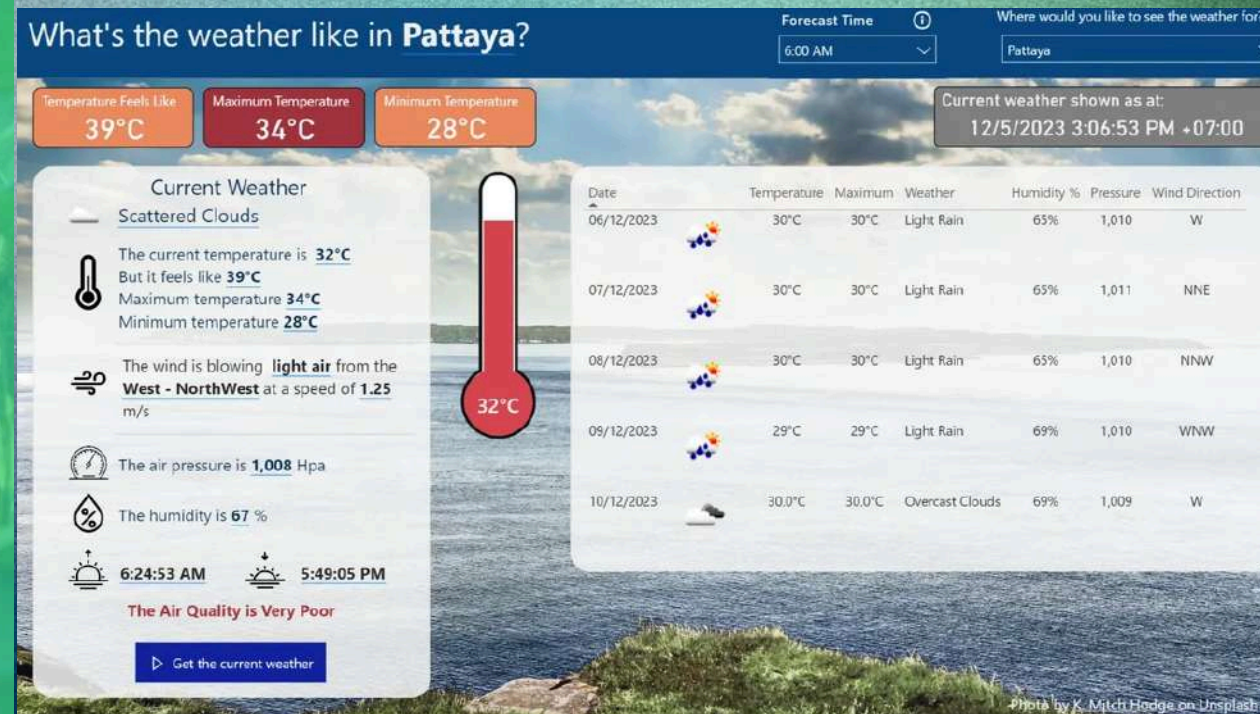
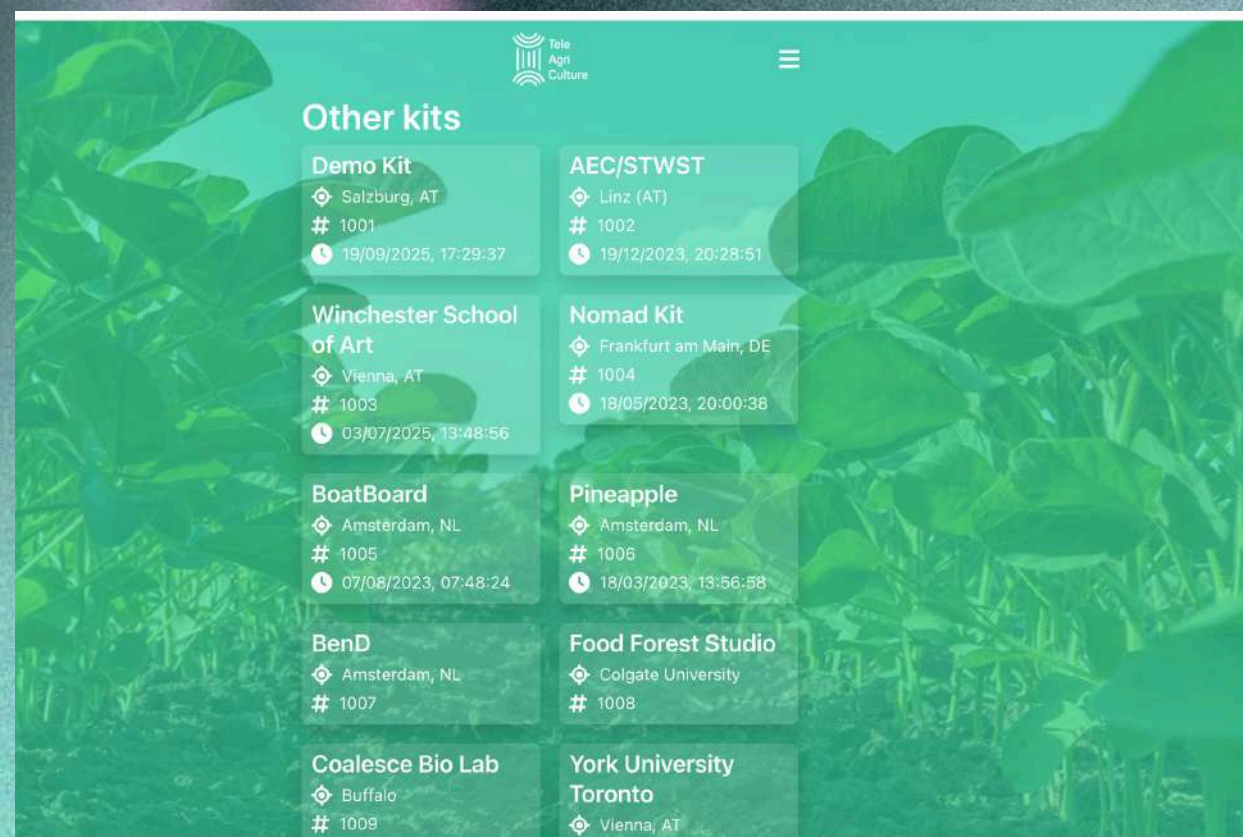




# Forecasting: 5-Day Indoor Prediction

## Forecast Workflow

- Pull 5-day OpenWeatherMap forecast
- Learn indoor/outdoor delta per board
- Project indoor temperature forward
- Flag future stress risks





# Intended user surveys & Community Impact

## Primary Users

- Community growers
- Urban agriculture projects
- TeleAgriCulture collaborators

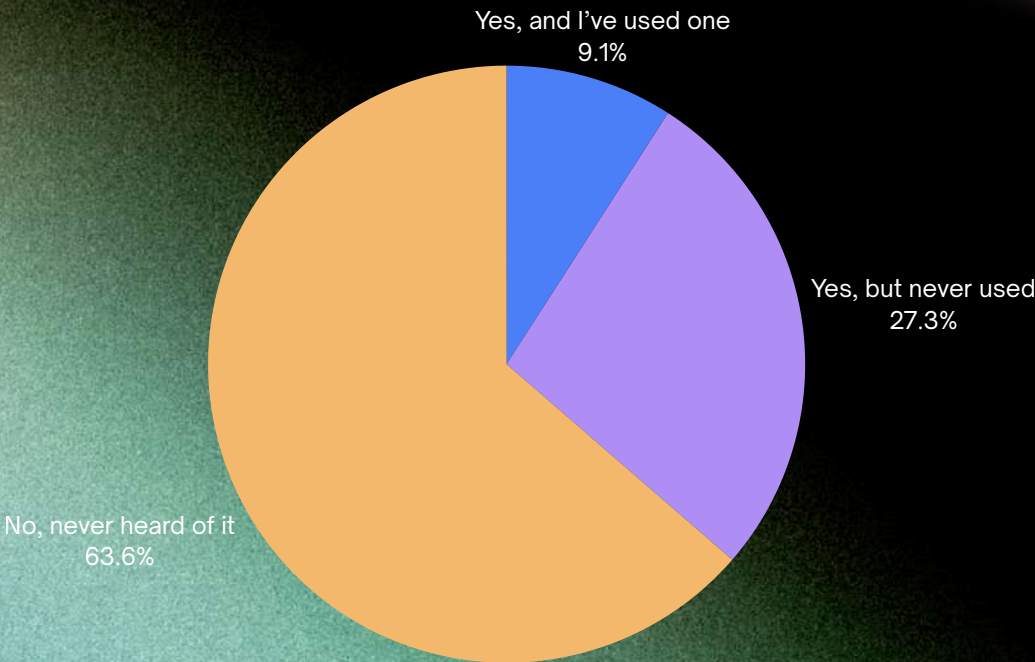


## Secondary Users

- Environmental educators
- Cultural institutions
- Climate researchers

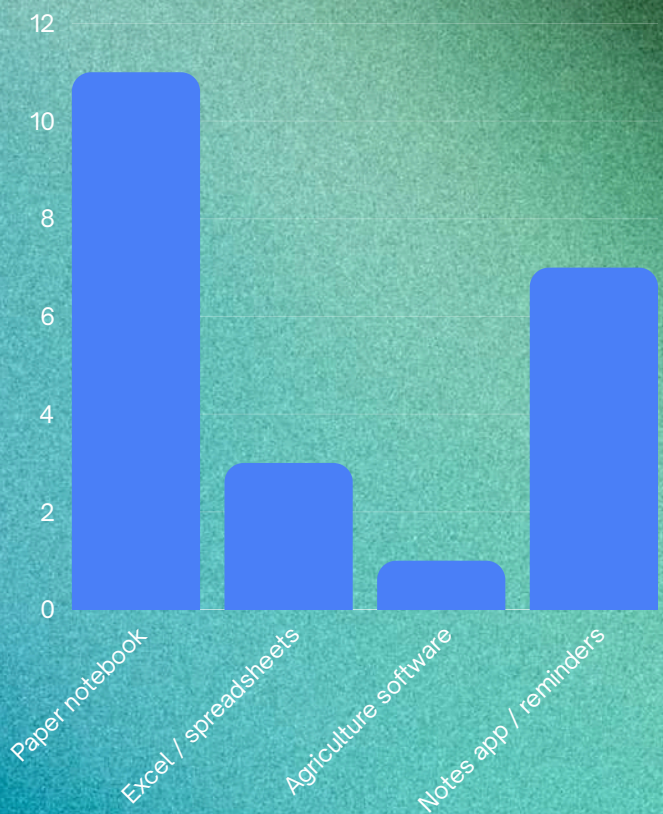
## Awareness of Agriculture Tracking Software

Question: "Have you heard of farm/agriculture tracking software?"



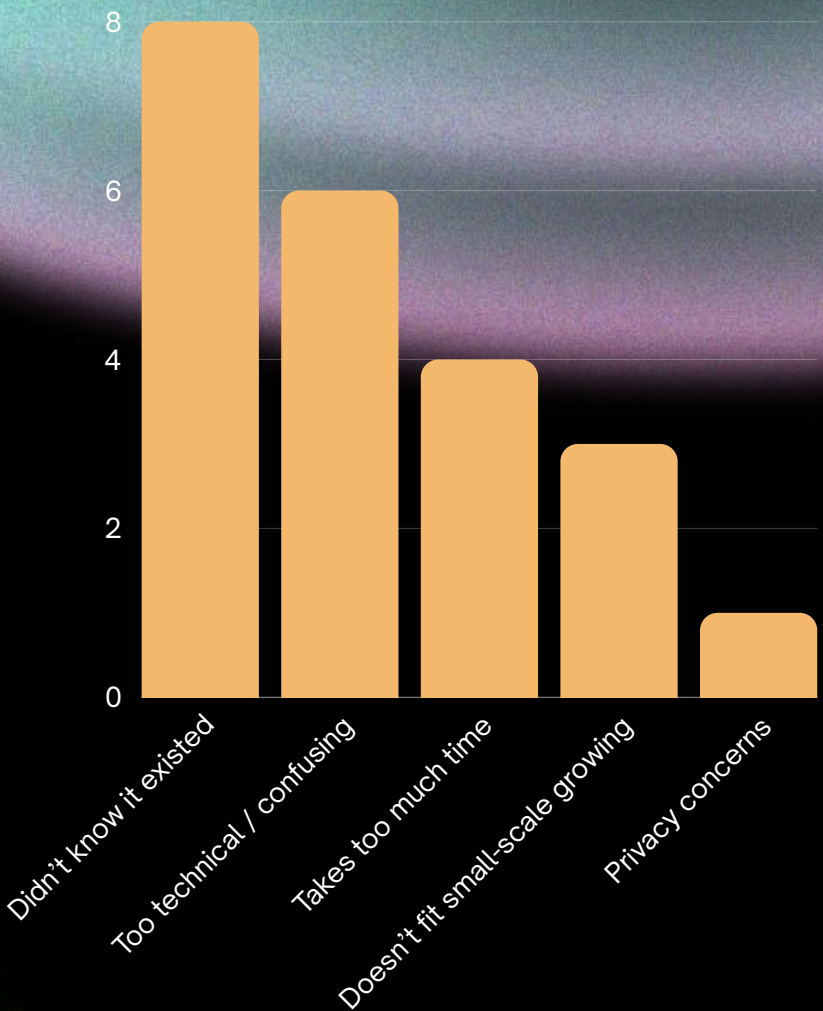
## What People Use Instead

Question: "What do you currently use to track growing conditions?"



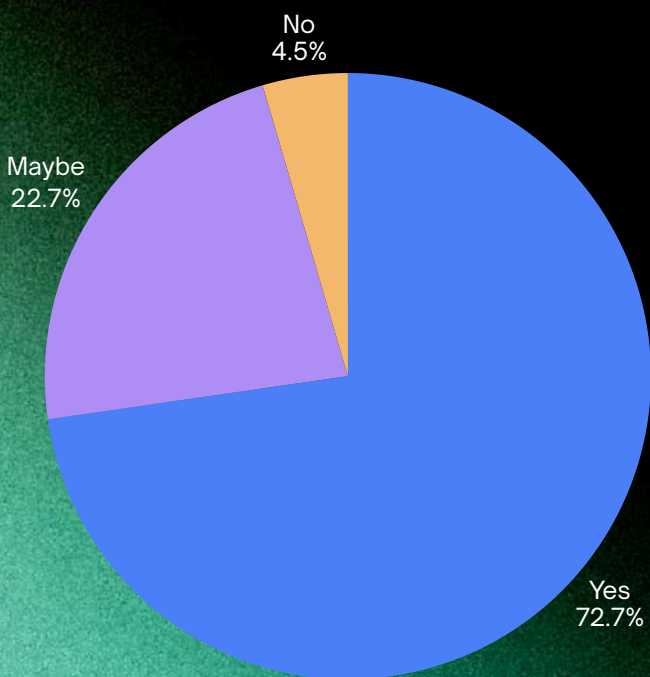
## Main Barriers

Question: "If you don't use agriculture software, why not?"



## Interest in Plain-Language Tool

Question: "Would you use a tool that gives summaries like 'heat stress' or 'drought risk'?"





# Intended user surveys ethics

Conducting these researchs needed an ethic analysis and assesments of which I have performed:

Before participation, I made sure that:

participants were given a short explanation of the project and its aims:

- it was clear that participation was voluntary
- participants could stop or skip questions at any time
- no personal identifying information was collected

For the pilot survey specifically (n = 22), responses were:

- collected anonymously through informal conversations and Google Forms
- used only to inform design decisions and presentation material for this project
- summarised into aggregate statistics rather than individual responses
- not shared publicly outside of the module submission and portfolio documentation

The survey focused on participants' awareness and use of environmental or agriculture-related tracking tools. Questions were kept simple and low risk, with no collection of sensitive personal data.

The findings were used to support decisions around:

- interface accessibility
- plain-language summaries
- interpretable AI outputs
- reducing technical barriers for non-expert users

There were no vulnerable participants involved, and all participants were over 18.

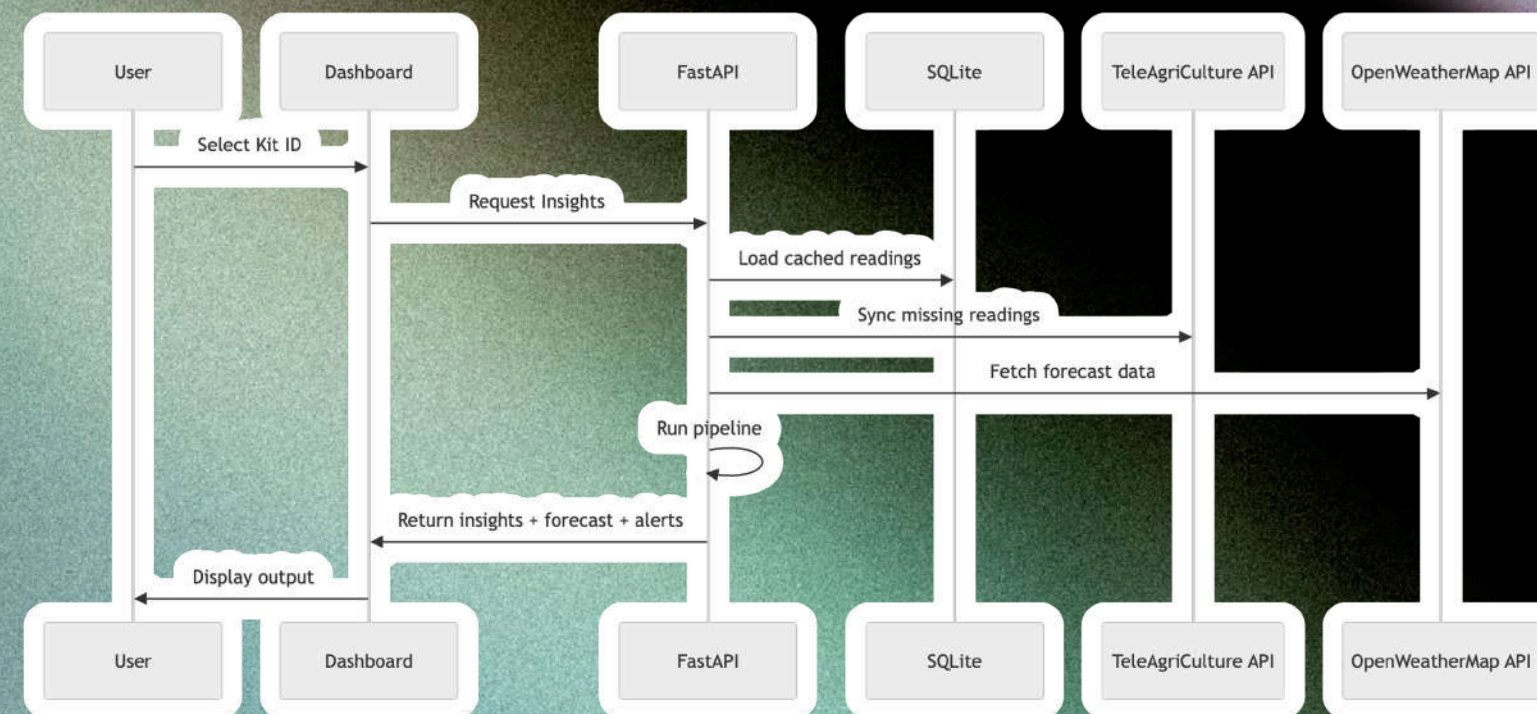
Overall, the research approach prioritised:

- informed consent
- anonymity in reporting
- minimal data collection
- low risk participation
- responsible use of participant feedback

This was important because the project explores environmental data and AI-assisted insight generation, where transparency and trust are central to the intended user experience.



# Survey pt2: Shaping the UI



HOME MY DATA CLIMATE DEBT

## LOCATION SURVEILLANCE

SOUTHAMPTON COMMON PARK, SOUTHAMPTON

AVG TEMP  
12.5°C

AVG HUMIDITY  
75%

AVG NOISE  
66 dB

## TEMPERATURE DECAY TIMELINE



## AgriSense Commons

TeleAgriCulture · Forecast & Alert Intelligence

Kit 1001 · live

KIT ID LAT LON SENSOR Run pipeline

12:19:41 UTC

Insights Forecast Alerts

### CURRENT CONDITIONS

OPTIMAL

23.5 °C

mean across 1,219 readings · 2019-08-05 - 2019-09-25

### TIME IN STATE

optimal 100%

### VS OUTDOOR (SALZBURG)

+13.6°C outdoor: 9.9°C · broken clouds

Temperature is in the ideal growing range. Conditions are favourable. Your readings are 13.6° warmer than outdoor ambient.

### DECISION TREE RULES

### BOARD PROFILE

|                       |            |
|-----------------------|------------|
| Kit ID                | #1001      |
| Sensor                | ftTemp     |
| Records               | 1,219      |
| Mean                  | 23.542 °C  |
| Std dev               | 1.542      |
| 25th pct              | 22.31      |
| 75th pct              | 24.44      |
| Date from             | 2019-08-05 |
| Date to               | 2019-09-25 |
| RECENT ANOMALIES (10) |            |
| optimal · 23.0852°C   | z=2.69     |
| optimal · 23.1585°C   | z=2.93     |



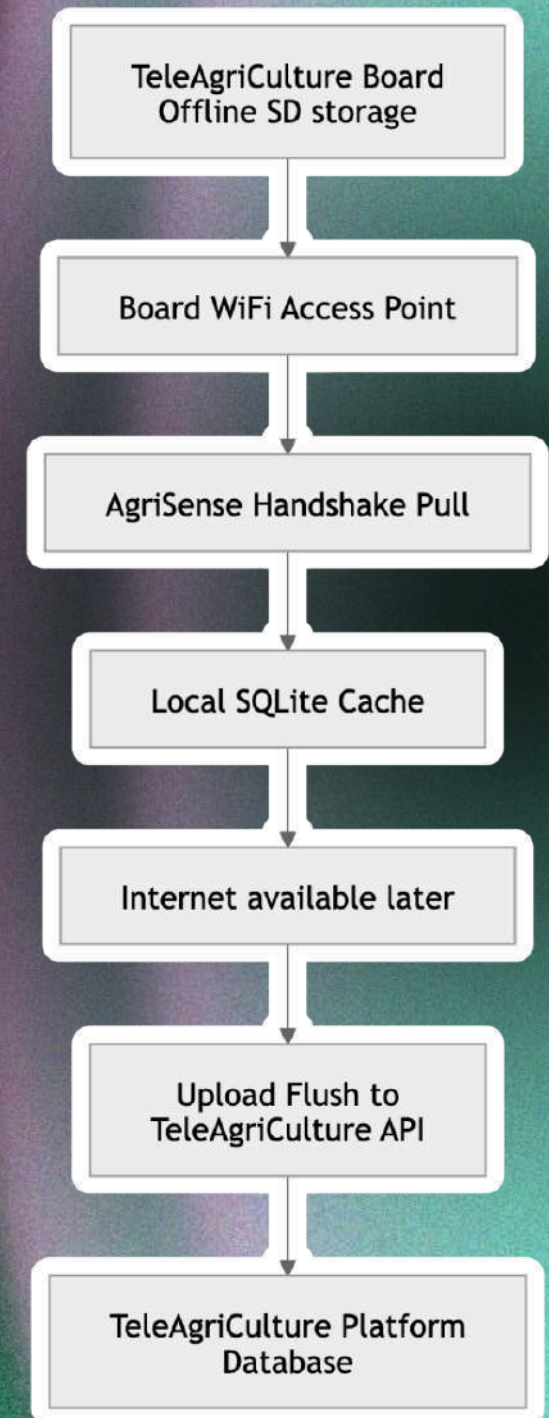
# Offline to Online design

- Board hosts its own WiFi access point
- AgriSense connects locally to pull cached readings
- Data stored in SQLite
- Upload flush sends readings to TeleAgriCulture later

```
env.example  pipeline.py M  dashboard.html  discovery.py X  .env
discovery.py > ...
75
76 def discover_boards_by_range(
77     start_id: int = 1000,
78     end_id: int = 1050,
79     on_found=None,
80 ) -> List[dict]:
81     """
82     Scan a range of Kit IDs to find active public boards.
83
84     Args:
85         start_id: first Kit ID to probe (inclusive)
86         end_id: last Kit ID to probe (inclusive)
87         on_found: optional callback called with (kit_id, sensors) when a board is found
88
89     Returns list of dicts: [(kit_id, sensors)]
90
91     Note: a range of 50 boards takes roughly 2-3 minutes due to API rate limiting.
92     For larger scans, run this as a background task.
93     """
94     found_boards = []
95
96     for kit_id_int in range(start_id, end_id + 1):
97         kit_id = str(kit_id_int)
98
99         # First check if the board exists at all with a single fast probe
100         if not probe_board(kit_id, "ftTemp"):
101             # Try one more sensor before giving up on this ID
102             if not probe_board(kit_id, "ftHumid"):
103                 time.sleep(0.2)
104                 continue
105
106         # Board exists - probe all sensors
107         sensors = discover_sensors_for_kit(kit_id)
108
109         if sensors:
110             entry = {"kit_id": kit_id, "sensors": sensors}
111             found_boards.append(entry)
112
```

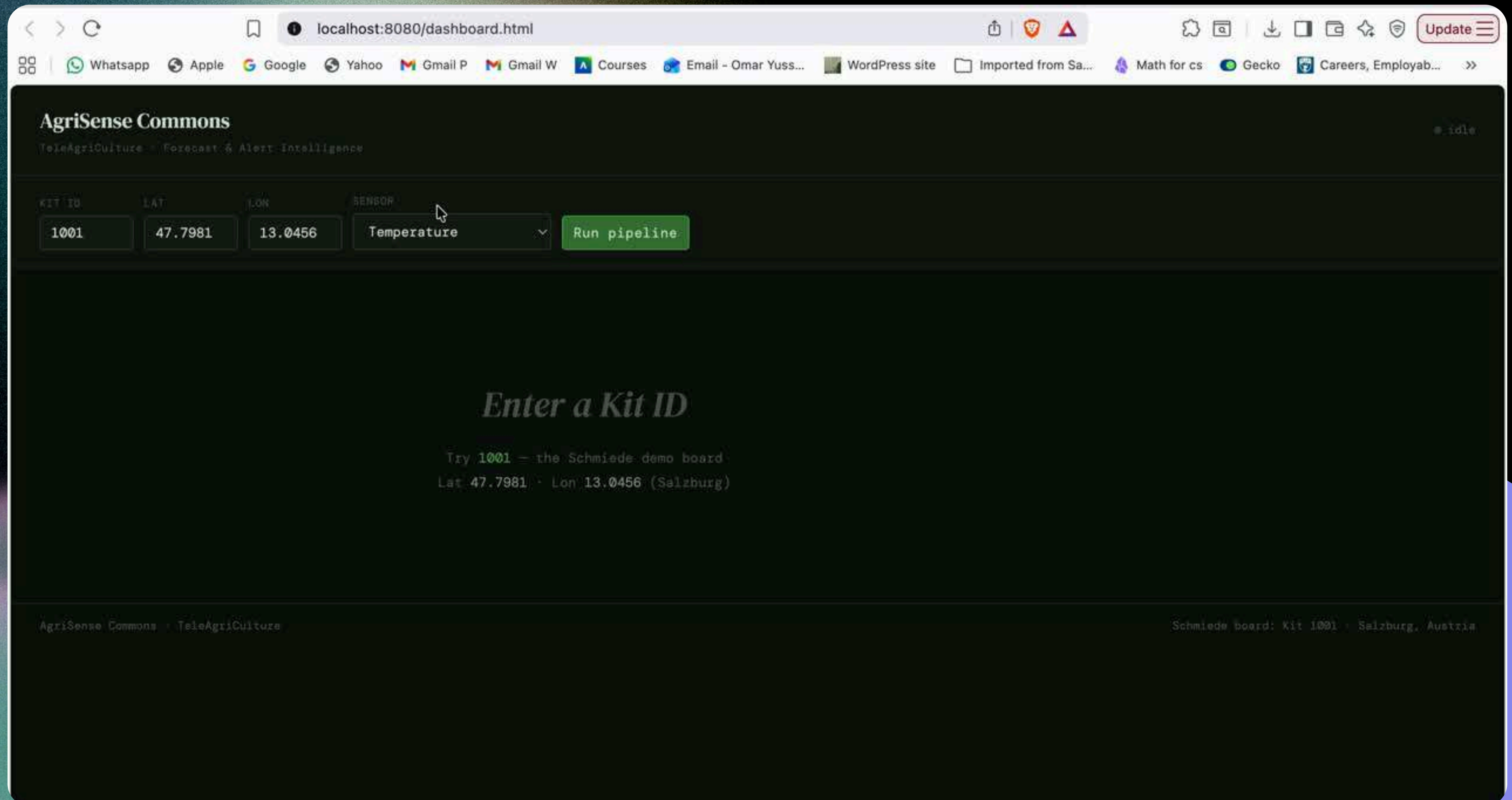
```
env.example  pipeline.py M  dashboard.html  discovery.py X  .env
discovery.py > ...
22 import requests
23 import time
24 from typing import Optional
25 from database import upsert_board, get_all_boards
26
27 TELEAGRI_BASE = "https://kits.teleagriculture.org/api/kits"
28
29 # Sensor names to probe per board - most common on TeleAgriCulture boards
30 PROBE_SENSORS = [
31     "ftTemp",
32     "ftHumid",
33     "ftSoilMoist",
34     "ftMoisture",
35     "ftLight",
36     "ftIDS",
37     "ftSoilTemp",
38     "ftPressure",
39     "ftADC",
40 ]
41
42 # Known active boards as starting seeds
43 SEED_KIT_IDS = ["1001", "1002", "1003", "1004", "1005"]
44
45
46 def probe_board(kit_id: str, sensor_name: str, timeout: int = 8) -> bool:
47     """
48     Check if a board+sensor combination has any data.
49     Returns True if the API returns at least one reading, False otherwise.
50     Does not raise - all errors return False.
51     """
52     try:
53         url = f"{TELEAGRI_BASE}/{kit_id}/{sensor_name}/measurements"
54         r = requests.get(url, timeout=timeout)
55         if r.status_code != 200:
56             return False
57         data = r.json()
58         return isinstance(data.get("data"), list) and len(data["data"]) > 0
59     except Exception:
60
```

```
env.example  pipeline.py M  dashboard.html  upload.py X  .env
upload.py > ...
97
98 def queue_readings_for_upload(kit_id: str, sensor_name: str, readings: List[dict]) -> int:
99     """
100     Add readings to the upload queue. Skips duplicates.
101     readings: List of dicts with keys created_at and value.
102     Returns number of new items queued.
103     """
104     if not readings:
105         return 0
106
107     rows = [
108         (kit_id, sensor_name, str(r["created_at"]), float(r["value"]))
109         for r in readings
110     ]
111
112     with_db() as conn:
113         before = conn.execute(
114             "SELECT COUNT(*) FROM upload_queue WHERE kit_id=? AND sensor_name=? AND status='pending'",
115             (kit_id, sensor_name)
116         ).fetchone()[0]
117
118         conn.executemany("""
119             INSERT OR IGNORE INTO upload_queue (kit_id, sensor_name, created_at, value, status)
120             VALUES (?, ?, ?, ?, 'pending')
121             """, rows)
122
123         after = conn.execute(
124             "SELECT COUNT(*) FROM upload_queue WHERE kit_id=? AND sensor_name=? AND status='pending'",
125             (kit_id, sensor_name)
126         ).fetchone()[0]
127
128     return after - before
129
130
131 def get_pending_uploads(kit_id: str, sensor_name: str, limit: int = 100) -> List[dict]:
132     """Return pending upload queue items for a board, oldest first."""
133     with_db() as conn:
134         rows = conn.execute("""
135
```





# Final project merge

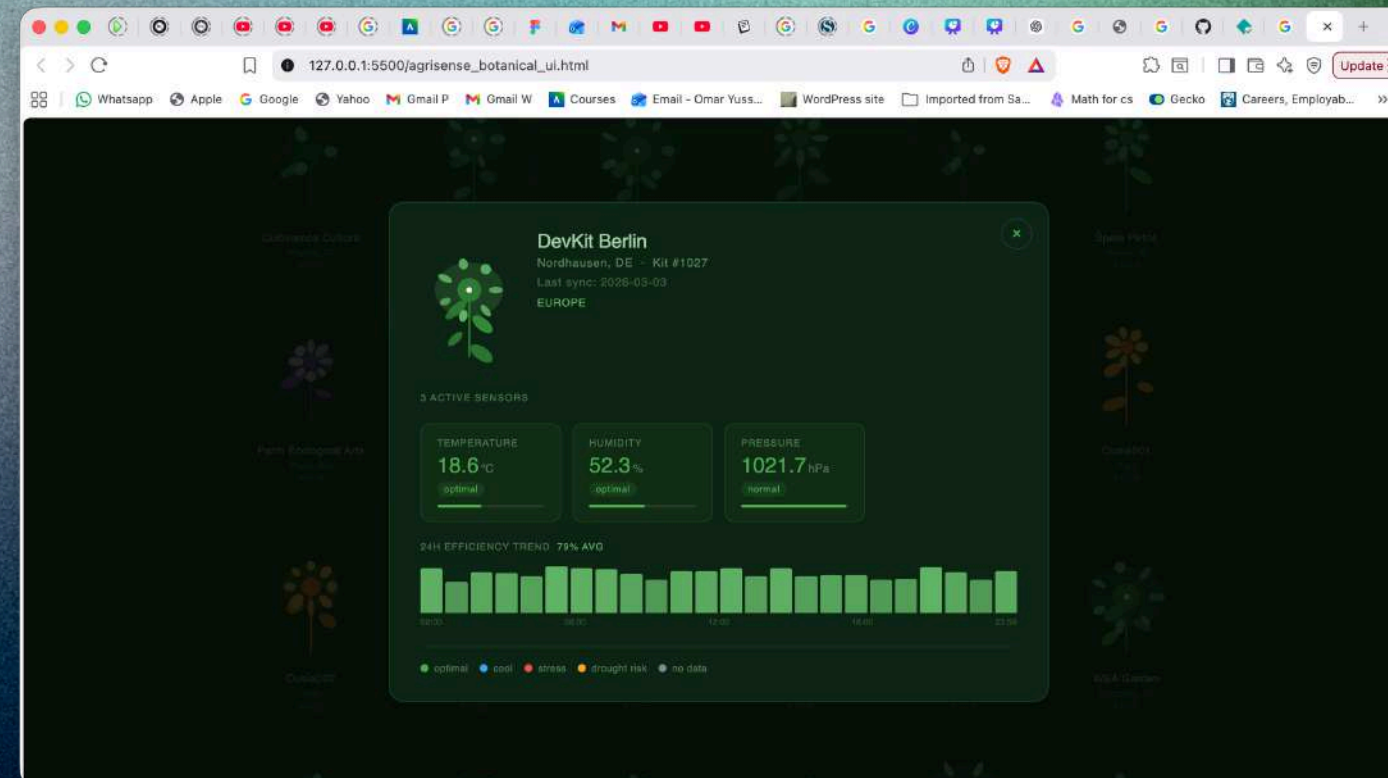
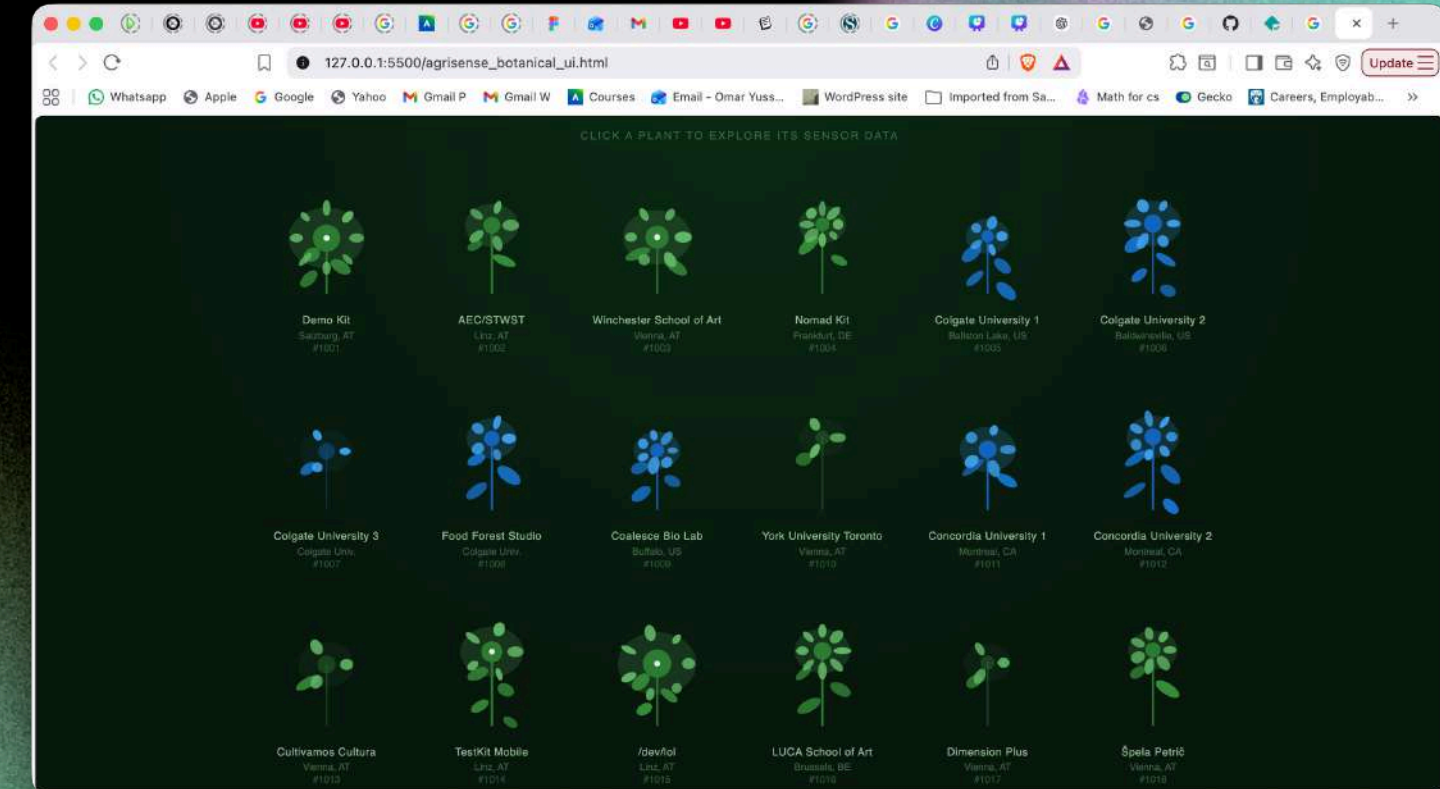




# Agri-sense 2.0

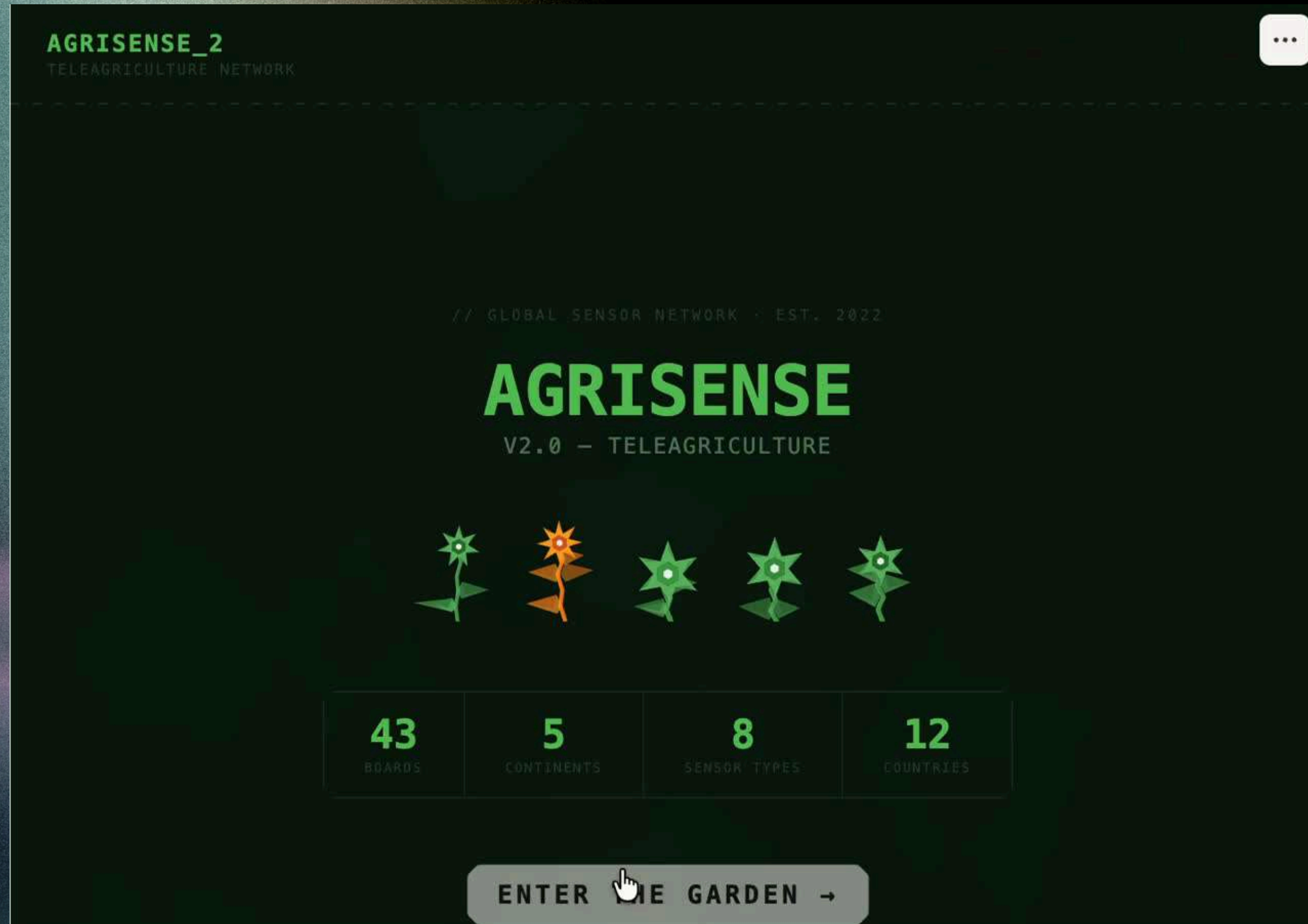
## Updates implemented:

- SQLite into JSON output
- Refined the z-cores for anomaly detection (for data crash prevention once imported)
- Simplified the feature engineering more and prioritised HCI conveyance over bloatware
- New artistic UI prototype inspired by Grow Home (Ubisoft, 2015)





# Final Artistic DataViz





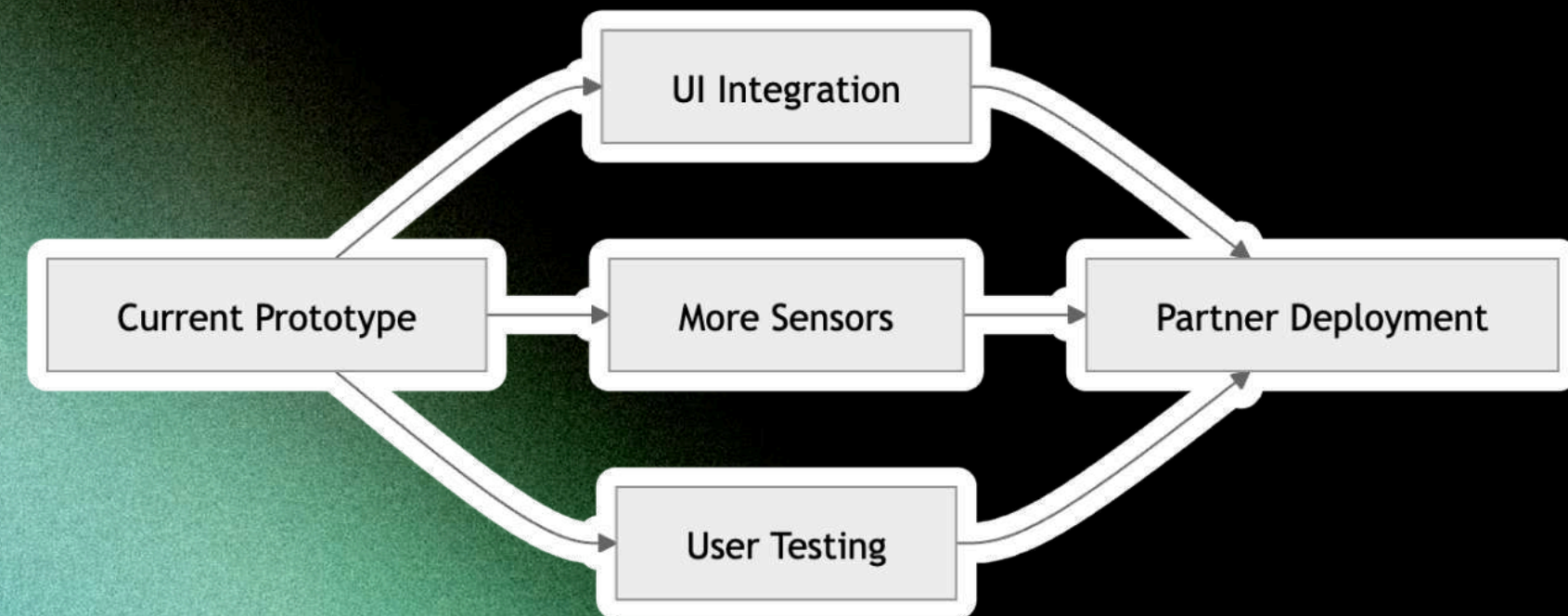
# Reflection & Next Steps

What went well:

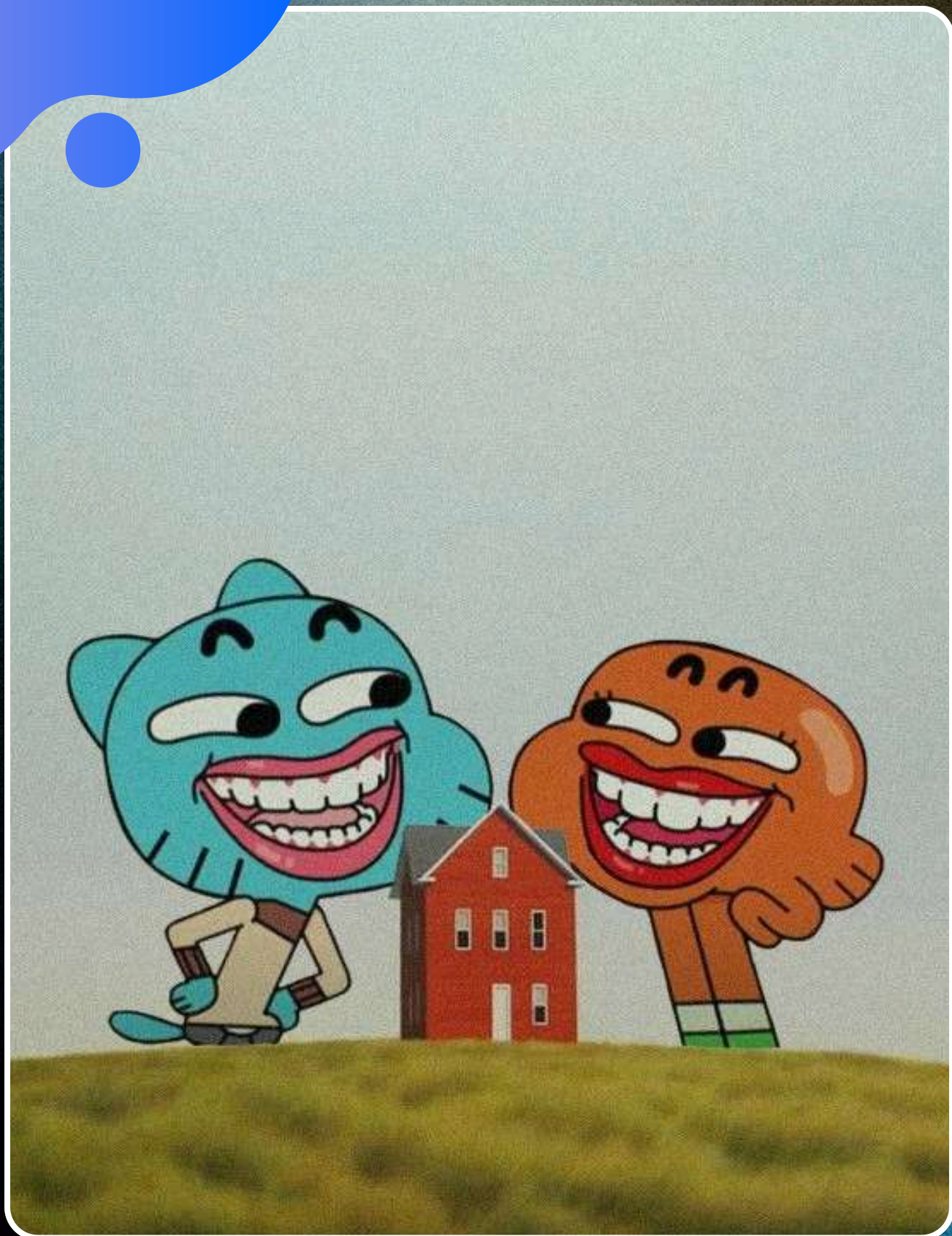
- Interpretable ML pipeline
- Modular architecture and documentation
- Offline data capture workflow
- Forecasting integration

Scalability:

- Expand beyond temperature and soil moisture into pollution and toxicity datasets
- Improve dashboard into full React UI
- Conduct structured user testing with growers







**Thank You!**